

Analysis: Banana Bunches

Formally the problem can be written as: given an array of size N , select at most two non-overlapping subarrays, such that, sum of all the selected elements is equal to K . We need to minimise the sum of lengths of these two subarrays. We can calculate the prefix sum of the given array in $O(N)$. Pre_i denotes the sum of all the elements from 1 to i . The sum of elements in a particular $subarray(i, j)$ is given by $Pre_j - Pre_{i-1}$ and can be calculated in $O(1)$ time.

One special case is when there is an element equal to K in the given array. In this case, we can return 1 as the answer and can be done in a $O(N)$ traversal. For all the other cases, we can get the answer by selecting two non-overlapping subarrays.

Test set 1

For this test set, we can consider all possible pair of subarrays as constraints on N are small. The total number of pair of subarrays are $O(N^4)$. For each pair of non-overlapping subarrays, one can check whether the sum of elements in these subarrays is equal to K . Formally, consider all (i, j, x, y) such that sum of $subarray(i, j)$ and $subarray(x, y)$ is equal to K and $i \leq j < x \leq y$. Among all such pairs of subarrays, the optimal answer is the pair with the minimum sum of length of the subarrays. Formally, we need to minimise $j - i + 1 + y - x + 1$ over all such pairs. For each pair, one can calculate the sum and length of these subarrays in $O(1)$. Hence, the overall complexity of the solution is $O(N^4)$.

Test set 2

For this test set, we cannot consider all possible pair of subarrays as this solution would time out with the given constraints. We can iterate on all possible (i, j, x) and find an optimal index y such that sum of $subarray(i, j)$ and $subarray(x, y)$ is equal to K and $i \leq j < x \leq y$. For a particular (i, j, x) we need to find smallest index y such that $Pre_j - Pre_{i-1} + Pre_y - Pre_x - 1 = K$. This can be done by performing a binary search on indexes from x to N . If there is no such index y , we can ignore this triplet. Among all such possible (i, j, x, y) quadruples, we need to minimise $j - i + 1 + y - x + 1$. We are able to find an optimal index l in $O(\log N)$ time complexity for a particular triplet (i, j, x) . There are $O(N^3)$ such triplets. Hence, the overall complexity of the solution is $O(N^3 \log N)$.

We can further reduce the complexity by using two pointers instead of binary search. We can consider all possible (j, x) and then use two pointer approach to find optimal y for each i . For each pair (j, x) , we perform $O(N)$ operations. Hence, the overall complexity of the solution is $O(N^3)$.

Test set 3

For this test set, we can store the optimal second subarray length for each sum and iterate over each possible first subarray.

Consider the following pseudocode:

```
for i = 0 to K:
    Best[i] = inf
ans = inf
for i = N to 1:
    currSum = 0
    for j = i to 1:
        // |currSum| denotes sum of subarray(j, i).
        currSum += B[j]
        if currSum <= K:
            // Best[K - currSum] denotes the minimum length of subarray starting after index i which has sum equ
            ans = min(ans, j - i + 1 + Best[K - currSum])

    currPostSum = 0
    for x = i to N:
        // |currPostSum| denotes sum of subarray(i, x).
        currPostSum += B[x]
        if currPostSum <= K:
            // Update the minimum length of subarray with sum equal to |currPostSum|.
            Best[currPostSum] = min(Best[currPostSum], x - i + 1)
```

When we are iterating from N to 1 and are at index i currently, $Best_S$ stores the minimum length of subarray starting at any index greater than i and having sum S . Basically, this would denote the optimal length of second subarray with sum S if we choose first subarray ending at index i . We consider all indexes j from 1 to i . If sum of subarray (j, i) is $currSum$ (such that $currSum \leq K$), we update the answer. Then, we update the minimum length for each sum for the subarrays starting at index i . We do this by iterating for x from i to N . If sum of $subarray(i, x)$ is $currPostSum$, then update $Best_{currPostSum} = \min(Best_{currPostSum}, x - i + 1)$. As $K \leq 10^6$, we can maintain an array for storing $Best$. This would allow updating and looking up the sum in $O(1)$ time. For each index we do $O(N)$ operations, hence the overall complexity is $O(N^2 + K)$.

Analysis: Dogs and Cats

We need to determine if all the dogs will be able to eat food given the condition that an animal can only eat if the animal before it has already eaten. We do not worry if some cats are hungry as long as all the dogs are able to eat. Let us assume that the last (or rightmost) dog is at position p . We do not need to worry about the cats in positions $[p + 1, N]$. Since an animal can only eat food if the animal before it has already eaten, we need to make sure that all cats and dogs in positions $[1, p]$ are able to eat food.

Test Set 1

We are given $M = 0$. This means whenever a dog eats food, no cat foods magically appear. If S contains all 0 i.e., no dog is present, the answer is YES. Otherwise, we find the position of the last (or rightmost) dog, p . Let the total number of dogs i.e., the number of 1 in S be X and the number of cats before the rightmost dog i.e., the number of 0 in $[1, p - 1]$ be Y . The answer is YES if $D \geq X$ and $C \geq Y$ and NO otherwise.

Complexity : $O(N)$ per test case

Test Set 2

We are given $M \geq 0$. This means whenever a dog eats food, M cat food portions are added. We iterate over positions $[1, N]$ and whenever we see a cat we feed it with the cat food i.e., subtract 1 from C and whenever a dog appears we feed it with the dog food, i.e., subtract 1 from D . We need to ensure that the dog can only eat if the animal before it has already eaten, and to do so, we first make sure that $D \geq 1$, as 1 food item is needed to feed this dog and $C \geq 0$. If this is not true, our answer is NO. Otherwise we subtract 1 from D and add M to C . If the above condition is satisfied for all the dogs, our answer is YES. Note that this algorithm will also work for Test Set 1 but not vice versa.

Sample Code(C++)

```
bool have_dogs_eaten(string S, int N, int D, long long C, int M) {
    for(int i = 0; i < N; i++) {
        if (S[i] == '1') {
            if(D <= 0 || C < 0) {
                return false;
            }
            D--;
            C += M;
        } else {
            C--;
        }
    }
    return true;
}
```

Note that variable C is long long since maximum possible value does not fit in 32-bit data type.

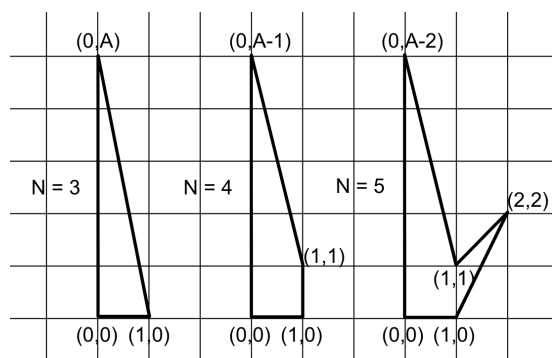
Complexity : $O(N)$ per test case

Analysis: Simple Polygon

According to [Pick's theorem](#), the area of a simple polygon having integer vertex coordinates is $Area = i + \frac{b}{2} - 1$, where i is the number of integer points inside the polygon and b is the number of integer points on its border. If we double this area, as in our problem statement, it follows that $\mathbf{A} = 2 \times Area = 2i + b - 2$. Since $i \geq 0$ and $b \geq \mathbf{N}$, a lower bound on the 'doubled-area' $\mathbf{A}$ of a polygon with $\mathbf{N}$ vertices is $\mathbf{A} = 2i + b - 2 \geq \mathbf{N} - 2$. Therefore, if $\mathbf{A} < \mathbf{N} - 2$, the answer is IMPOSSIBLE. In what follows, we will show that this is a [tight](#) lower bound by constructing an $\mathbf{N}$ vertex simple polygon having a 'doubled-area' $\mathbf{A}$ for any given $\mathbf{A} \geq \mathbf{N} - 2$.

Test Set 1

There are many ways to construct the necessary polygons. The following drawing shows possible constructions for $3 \leq \mathbf{N} \leq 5$.

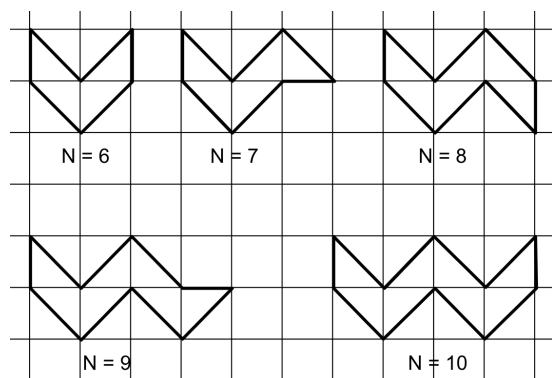


These polygons have no internal integer points, therefore, by Pick's theorem, their 'doubled-area' is $b - 2$. For example, for $\mathbf{N} = 5$, we can verify that $b = \mathbf{A} + 2$ by counting the integer points on the border. Therefore, the 'doubled-area' is $b - 2 = \mathbf{A} + 2 - 2 = \mathbf{A}$, which validates our construction. Similarly, it can be verified that we have achieved the desired area for $\mathbf{N} = 3$ and $\mathbf{N} = 4$ as well.

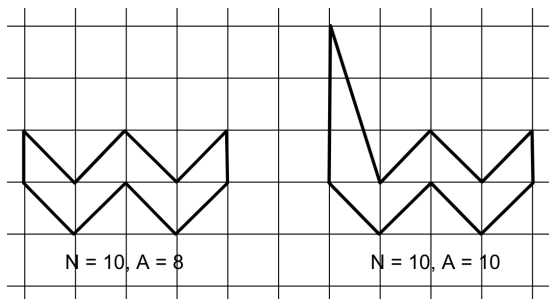
The time complexity of the construction is $O(1)$.

Test Set 2

For $\mathbf{N} > 5$, the construction is a little more involved. Let us start with the base case, where the 'doubled-area' of the polygon is the smallest possible, namely, $\mathbf{N} - 2$. The following drawing illustrates the construction for $6 \leq \mathbf{N} \leq 10$, but it can be generalized for arbitrary $\mathbf{N}$ by extending the zig-zag shape to the right.



The base polygon has $\mathbf{N}$ integer points on the border and no internal integer points, therefore, its 'doubled-area' is $\mathbf{N} - 2$. If $\mathbf{A} > \mathbf{N} - 2$, we just need to introduce $\mathbf{A} - \mathbf{N} + 2$ more points on the border by say, lifting the top-left vertex up $\mathbf{A} - \mathbf{N} + 2$ units as shown in the following drawing for $\mathbf{N} = 10$ and $\mathbf{A} = 10$.



The time complexity of the construction is $O(\mathbf{N})$.

Analysis: Staying Hydrated

The [Manhattan distance](#) between a point $P = (x, y)$ and an axis-parallel rectangle R with the lower-left corner (x_1, y_1) and the upper-right corner (x_2, y_2) is $d(P, R) = \max(x_1 - x, x - x_2, 0) + \max(y_1 - y, y - y_2, 0)$. Intuitively, the first term represents the minimal horizontal movement from P to the nearest point inside the x -interval $[x_1, x_2]$, which could be 0 if P itself is in that interval. Similarly, the second term represents the minimal vertical movement from P to the nearest point inside the y -interval $[y_1, y_2]$.

Test Set 1

In this test set, the furnished area of the room is so small that we can afford testing every point $P = (x, y)$ in the range $-100 \leq x, y \leq 100$ and select the point with the smallest total distance to all rectangular objects. The time complexity of such a brute-force algorithm is $O(\mathbf{K}WH)$, where $\mathbf{K}$ is the number of input objects and W and H is the width and the height of the smallest axis-parallel rectangle covering all input rectangles. Note that it is not necessary to test any points outside this covering rectangle as moving from such a point towards the covering rectangle would reduce the distance to all input rectangular objects. The time complexity can be further improved to $O(\mathbf{K}(W + H))$ if we realize that the optimal x - and y -coordinates are in fact independent and can be calculated separately.

Test Set 2

In this test set, the furnished area of the room may be huge, therefore, even the improved version of the brute-force algorithm would be too slow. However, with a little bit of sorting, we can find the optimal location of the bottle without ever computing any distances.

Let us consider finding the optimal x -coordinate of the bottle (the optimal y -coordinate can be found in the same way). Imagine that we are seeking the optimal location by sweeping the coordinate plane from left to right. Suppose we are currently at the position x and let $a(x)$ be the number of rectangles that are strictly ahead of x , i.e. $x < x_1$, where x_1 is the left coordinate of the rectangle. Similarly, let $b(x)$ be the number of rectangles that are behind x in the non-strict sense, namely, $x_2 \leq x$. Now, if $a(x) > b(x)$, then x is not the optimal location as moving one step to the right would reduce the total horizontal distance to the rectangles by $a(x) - b(x) > 0$.

What happens to the value of $a(x) - b(x)$ as we sweep the plane from left to right? For a sufficiently small x , which is strictly to the left of all rectangles, $a(x) = \mathbf{K}$ and $b(x) = 0$, and therefore, $a(x) - b(x) = \mathbf{K} > 0$. Conversely, for a sufficiently large x , it is the other way around and $a(x) - b(x) = -\mathbf{K} < 0$. And since $a(x)$ is a decreasing function while $b(x)$ is an increasing function, the difference $a(x) - b(x)$ is also a decreasing function. What does it mean for our plane sweeping approach? As long as $a(x) - b(x) > 0$, we should keep moving to the right as, by doing so, we are reducing the total distance to the rectangles. But as soon as $a(x) - b(x) \leq 0$, we have found the optimal x -coordinate, since further reduction of the total distance is not possible.

In summary, we have reduced the task of calculating the optimal Manhattan distance to finding the smallest x with $a(x) - b(x) \leq 0$. This can be done by iterating through the left and right coordinates of rectangles in sorted non-decreasing order and maintaining the values of functions $a(x)$ and $b(x)$. The time complexity of this algorithm is dominated by the sorting, and is therefore $O(\mathbf{K} \log \mathbf{K})$.

Left as an exercise to the reader: This problem can also be solved in $O(\mathbf{K})$ by using [linear-time selection algorithms](#). Though both our test sets should pass with the above $O(\mathbf{K} \log \mathbf{K})$ approach.